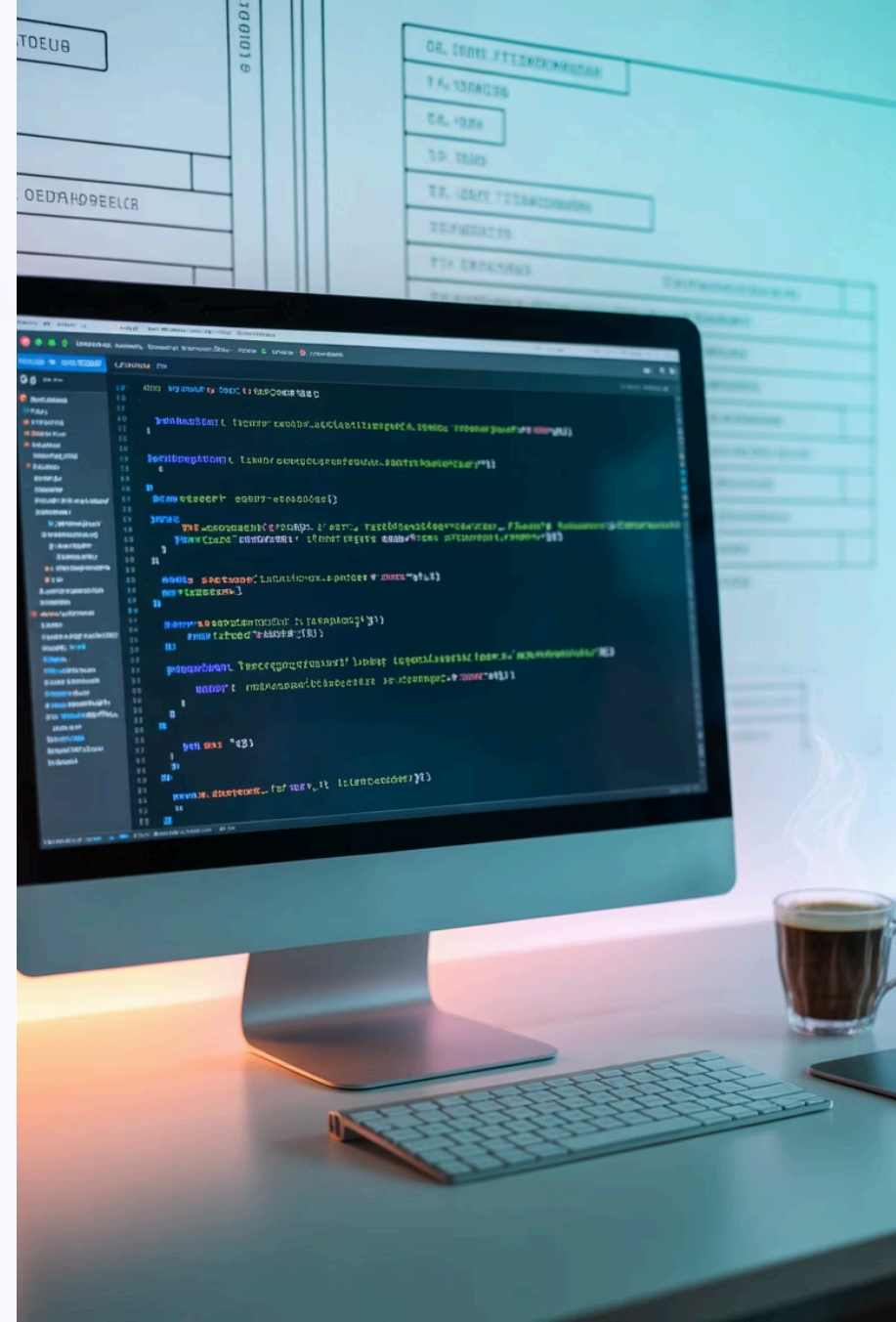


Anotaciones JPA Estándar para Mapeo de Entidades

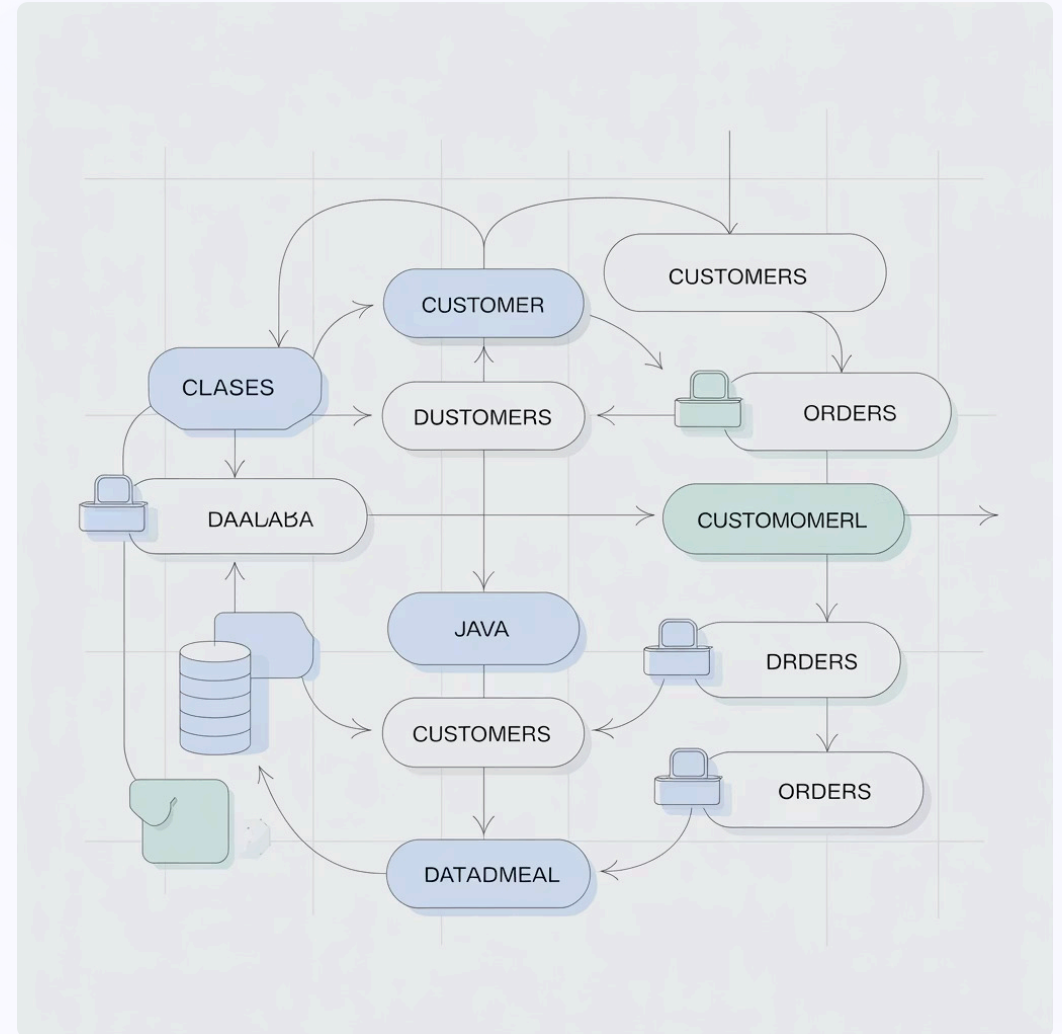
Esta presentación explora las anotaciones fundamentales de JPA (Java Persistence API) que permiten mapear clases Java a tablas de bases de datos relacionales, facilitando la persistencia de objetos de manera eficiente y declarativa.



¿Qué es JPA?

Java Persistence API (JPA) es una especificación de Java que describe la gestión de datos relacionales en aplicaciones Java. Permite:

- Mapear clases Java a tablas de bases de datos
- Persistir objetos automáticamente
- Consultar entidades mediante un lenguaje propio
- Configurar relaciones entre entidades





@Entity y @Table

1

@Entity

Declara que una clase Java es una entidad persistente, permitiendo que sus instancias sean guardadas y recuperadas de una base de datos.

```
@Entity
public class Producto {
    // ...
}
```

2

@Table

Especifica el nombre de la tabla en la base de datos a la que se mapea la entidad. También permite definir restricciones de unicidad.

```
@Entity
@Table(name = "productos_inventario")
public class Producto {
    // ...
}
```



Definición de Clave Primaria

@Id

Marca el atributo que actúa como la clave primaria de la entidad. Toda entidad debe tener una clave primaria.

```
@Id  
private Long id;
```

@GeneratedValue

Define cómo se generará el valor de la clave primaria. Es casi siempre usada junto con @Id.

Estrategias comunes:

- GenerationType.IDENTITY: La base de datos genera automáticamente el ID
- GenerationType.AUTO: El proveedor de JPA elige la estrategia más adecuada

```
@Id  
@GeneratedValue(strategy =  
    GenerationType.IDENTITY)  
private Long id;
```

@Column: Personalización de Columnas

Propósito

Personaliza el mapeo de un atributo de la entidad a una columna de la tabla. Permite especificar el nombre de la columna, si es nula, su longitud, etc.

Ubicación

A nivel de campo o método getter en la clase de entidad.

Ejemplo

```
@Column(name =  
    "nombre_producto",  
        nullable = false, length = 100)  
private String nombre;  
  
@Column(unique = true)  
private String codigoBarra;
```

Los atributos más comunes incluyen: name, nullable, length y unique, que permiten un control preciso sobre la estructura de la tabla.

Atributos de @Column

Atributo	Descripción	Ejemplo
name	Nombre de la columna en la DB (por defecto, usa el nombre del atributo)	name = "nombre_producto"
nullable	true (permite nulos) o false (no permite nulos, NOT NULL)	nullable = false
length	Longitud para columnas de tipo String (ej. VARCHAR(255))	length = 100
unique	true si los valores de la columna deben ser únicos	unique = true

@Transient: Campos No Persistentes

Propósito

Indica que un atributo de la entidad no debe ser persistido en la base de datos. Es útil para campos calculados o temporales que solo existen en el contexto de la aplicación.

Ubicación

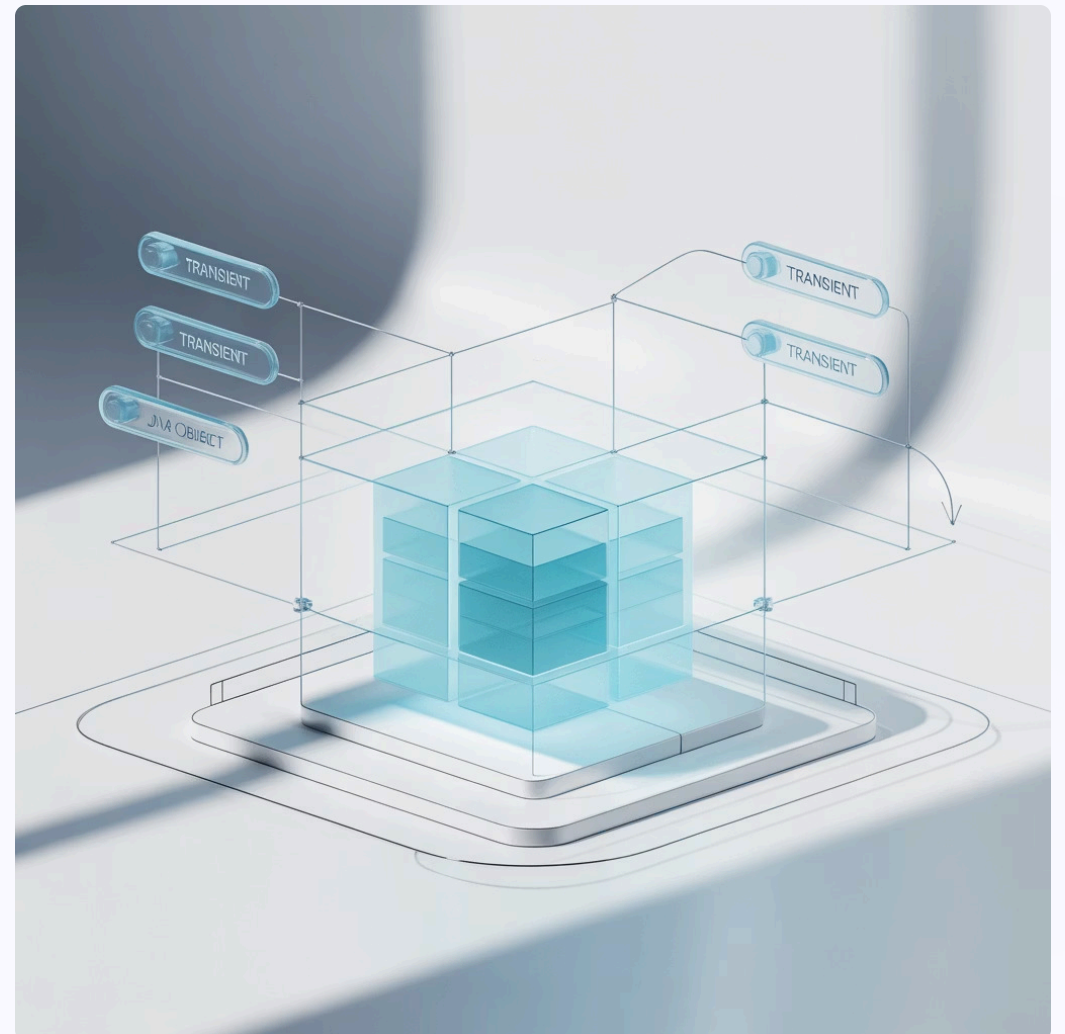
A nivel de campo o método getter.

Ejemplo

```
@Entity
public class Producto {
    @Id
    private Long id;

    private String nombre;

    @Transient
    private int stockDisponibleCalculado;
    // Este campo no se guardará en la DB
}
```



@Enumerated: Mapeo de Enumeraciones

Propósito

Especifica cómo un atributo de tipo enum se mapea a la base de datos.

Estrategia Recomendada

EnumType.STRING: Almacena el nombre del valor enum como un String (ej. "ACTIVO", "INACTIVO"). Altamente recomendado por su robustez.

Ejemplo

```
public enum EstadoPedido {  
    ACTIVO, INACTIVO, PENDIENTE  
}
```

```
@Enumerated(EnumType.STRING)  
private EstadoPedido estado;
```

Dacbbase 3

Detalle cliente D

Status	Priority
Open	Closed
Open	Closed
Open	Low
High	Medium
High	Low
High	Low

TaskStatus

Javi Tm

open

open

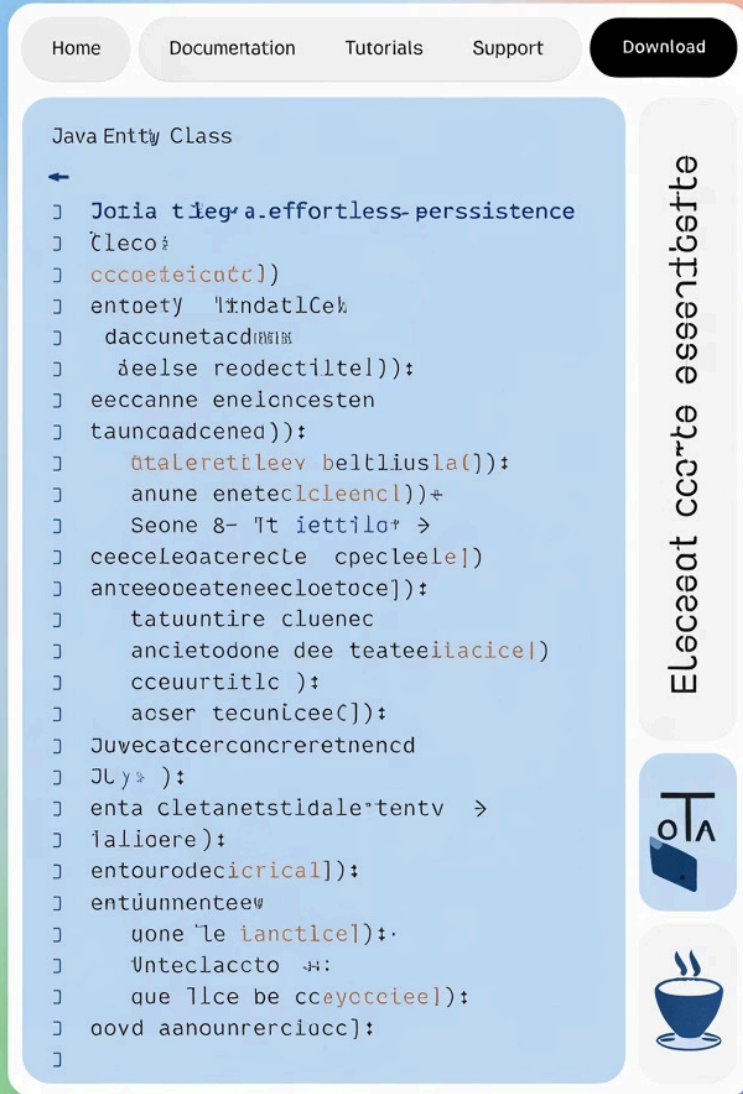
closed

pending

Enum Storage Explained

[Learn More](#)

Ejemplo Completo de Entidad JPA



```
@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre_producto", nullable = false, length = 100)
    private String nombre;

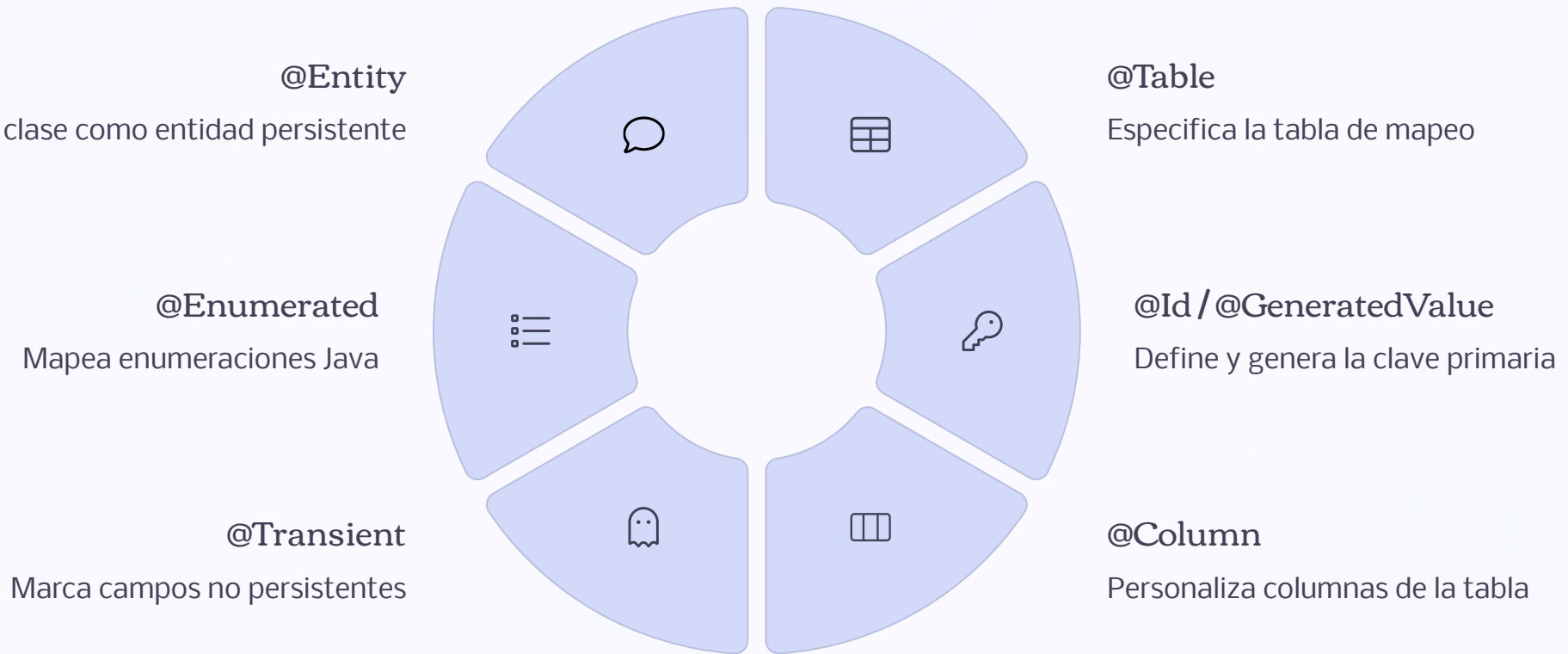
    @Column(unique = true)
    private String codigoBarra;

    @Enumerated(EnumType.STRING)
    private EstadoPedido estado;

    @Transient
    private int stockDisponibleCalculado;

    // Getters y setters
}
```

Resumen de Anotaciones JPA



Estas anotaciones proporcionan la base para el mapeo objeto-relacional en aplicaciones Java, permitiendo una integración fluida entre el código orientado a objetos y las bases de datos relacionales.